
```

clearvars; clc; close all;

% 1. SYSTEM SETUP & INITIALIZATION
m1 = 10; m2 = 10; % Mass (kg)
k1_healthy = 10e3; k1_damaged = 7.5e3; t_damage = 10; % Stiffness (N/m)
k2_true = 10e3; c1_true = 31.76; c2_true = 31.76; % Constant params

% Default choices for the main filter run
default_P_choice = 2;
default_Q_choice = 3;
X_init = [0; 0; 0; 0; 5000; 5000; 50; 50];
R_noise = diag([0.0438, 0.0967]);

% 2. DATA IMPORT & PRE-PROCESSING
meas_data = readmatrix('HW2_measurements.txt');
gm_data    = readmatrix('HW1_Q2_El_Centro.txt');

time_vec = meas_data(:, 1);
z_meas    = meas_data(:, 2:3)';
dt         = mean(diff(time_vec));
num_pts   = length(time_vec);

% Interpolate ground motion
ug_vec = interp1(gm_data(:,1), gm_data(:,2), time_vec, 'linear', 'extrap');

% 3. GENERATE TRUE REFERENCE TRAJECTORIES
X_true = zeros(8, num_pts);
X_true(5:8, 1) = [k1_healthy; k2_true; c1_true; c2_true];

for k = 1:num_pts-1
    k1_now = k1_healthy; k1_next = k1_healthy; k1_mid = k1_healthy;
    if time_vec(k) >= t_damage, k1_now = k1_damaged; end
    if time_vec(k+1) >= t_damage, k1_next = k1_damaged; end
    if time_vec(k) + 2*dt/3 >= t_damage, k1_mid = k1_damaged; end

    P_now = [k1_now; k2_true; c1_true; c2_true];
    P_mid = [k1_mid; k2_true; c1_true; c2_true];

    % Ralston integration for true state
    ug_now = ug_vec(k);
    ug_mid = interp1(time_vec, ug_vec, time_vec(k) + 2*dt/3, 'linear',
'extrap');

    f1 = calc_dynamics([X_true(1:4, k); P_now], ug_now, m1, m2);
    X2 = X_true(:, k) + (2*dt/3) * f1;
    f2 = calc_dynamics([X2(1:4); P_mid], ug_mid, m1, m2);

    X_true(:, k+1) = X_true(:, k) + dt * (0.25*f1 + 0.75*f2);
    X_true(5:8, k+1) = [k1_next; k2_true; c1_true; c2_true];
end

% 4. MAIN EKF EXECUTION

```

```

P0_main = get_P_matrix(default_P_choice);
Q_main = get_Q_matrix(default_Q_choice);
[X_hat_main, Z_hat_main] = execute_ekf_pass(X_init, P0_main, Q_main,
R_noise, time_vec, ug_vec, z_meas, dt, m1, m2);

% 5. VISUALIZATION - BASE RESULTS
set(groot, 'defaultAxesTickLabelInterpreter','latex');
set(groot, 'defaultLegendInterpreter','latex');

% Plot A: State Estimates
figure('Name', 'EKF State Tracking', 'Color', 'w', 'Position', [100, 100,
800, 600]);
tiledlayout(2,2,'TileSpacing','compact');
labels = {'$y_1$ (m)', '$y_2$ (m)', '$\dot{y}_1$ (m/s)', '$\dot{y}_2$ (m/
s)'};
for i = 1:4
    nexttile;
    plot(time_vec, X_true(i,:), 'k-', 'LineWidth', 1.5); hold on;
    plot(time_vec, X_hat_main(i,:), 'r--', 'LineWidth', 1.5);
    ylabel(labels{i}); xlabel('Time (s)'); grid on;
    if i == 1, legend('True', 'EKF Estimate', 'Location', 'best'); end
end

% Plot B: Parameter Estimates
figure('Name', 'EKF Parameter Tracking', 'Color', 'w', 'Position', [150,
150, 800, 600]);
tiledlayout(2,2,'TileSpacing','compact');
p_labels = {'$k_1$ (N/m)', '$k_2$ (N/m)', '$c_1$ (Ns/m)', '$c_2$ (Ns/m)'};
for i = 1:4
    nexttile;
    plot(time_vec, X_true(i+4,:), 'k-', 'LineWidth', 1.5); hold on;
    plot(time_vec, X_hat_main(i+4,:), 'r--', 'LineWidth', 1.5);
    ylabel(p_labels{i}); xlabel('Time (s)'); grid on;
    if i==1, xline(t_damage, 'b:', 'LineWidth', 1.5); end
end

% 6. SENSITIVITY ANALYSIS: EFFECT OF P0
figure('Name', 'Sensitivity: Initial Covariance P0', 'Color', 'w',
'Position', [200, 200, 900, 600]);
tiledlayout(2,2,'TileSpacing','compact');
colors = {'#0072BD', '#D95319', '#EDB120', '#7E2F8E'};
lines = {'-', '--', ':', '-.'};

for i = 1:4
    nexttile; hold on;
    plot(time_vec, X_true(i+4,:), 'k', 'LineWidth', 2);
    for p_idx = 1:4
        [X_temp, ~] = execute_ekf_pass(X_init, get_P_matrix(p_idx), Q_main,
R_noise, time_vec, ug_vec, z_meas, dt, m1, m2);
        plot(time_vec, X_temp(i+4,:), 'Color', colors{p_idx}, 'LineStyle',
lines{p_idx}, 'LineWidth', 1.2);
    end
    ylabel(p_labels{i}); xlabel('Time (s)'); grid on; xlim([0 5]); % Zoom in
to see early effect

```

```

        if i == 1, legend('True', 'Choice 1', 'Choice 2', 'Choice 3', 'Choice
4'); title('Early transient response heavily relies on P_0'); end
end

% 7. SENSITIVITY ANALYSIS: EFFECT OF Q
figure('Name', 'Sensitivity: Process Noise Q', 'Color', 'w', 'Position',
[250, 250, 900, 600]);
tiledlayout(2,2,'TileSpacing','compact');

for i = 1:4
    nexttile; hold on;
    plot(time_vec, X_true(i+4,:), 'k', 'LineWidth', 2);
    for q_idx = 1:3
        [X_temp, ~] = execute_ekf_pass(X_init, P0_main, get_Q_matrix(q_idx),
R_noise, time_vec, ug_vec, z_meas, dt, m1, m2);
        plot(time_vec, X_temp(i+4,:), 'Color', colors{q_idx}, 'LineWidth',
1.2);
    end
    ylabel(p_labels{i}); xlabel('Time (s)'); grid on;
    if i == 1, legend('True', 'Choice 1 (Q=0)', 'Choice 2 (Small Q)',
'Choice 3 (Large Q)'); end
end

% LOCAL FUNCTIONS (Filter Engine & Math Models)

function [X_hist, Z_hist] = execute_ekf_pass(x0, P0, Q, R, t, ug, z_meas,
dt, m1, m2)
    % Encapsulated EKF Engine
    N = length(t);
    X_hist = zeros(8, N);
    Z_hist = zeros(2, N);
    P = P0;
    X_hist(:, 1) = x0;

    % Initial Update
    [z0, H0] = compute_meas_and_jacobian(x0, m1, m2);
    K0 = P * H0' / (H0 * P * H0' + R);
    X_hist(:, 1) = x0 + K0 * (z_meas(:, 1) - z0);
    P = (eye(8) - K0 * H0) * P;
    P = 0.5 * (P + P'); % Guarantee symmetry

    for k = 1:N-1
        u_now = ug(k);
        u_mid = interp1(t, ug, t(k) + 2*dt/3, 'linear', 'extrap');

        % Predict (Ralston)
        f1 = calc_dynamics(X_hist(:, k), u_now, m1, m2);
        A1 = calc_analytical_jacobian(X_hist(:, k), m1, m2);

        X2 = X_hist(:, k) + (2*dt/3) * f1;
        f2 = calc_dynamics(X2, u_mid, m1, m2);
        A2 = calc_analytical_jacobian(X2, m1, m2);

        X_pred = X_hist(:, k) + dt * (0.25*f1 + 0.75*f2);

```

```

    F_mat = eye(8) + dt * (0.25*A1 + 0.75*(A2*(eye(8) + (2*dt/3)*A1)));

    P_pred = F_mat * P * F_mat' + Q;
    P_pred = 0.5 * (P_pred + P_pred');

    % Update
    [z_pred, H_mat] = compute_meas_and_jacobian(X_pred, m1, m2);
    Innov_cov = H_mat * P_pred * H_mat' + R;
    K_gain = (P_pred * H_mat') / Innov_cov;

    X_hist(:, k+1) = X_pred + K_gain * (z_meas(:, k+1) - z_pred);
    P = (eye(8) - K_gain * H_mat) * P_pred;
    P = 0.5 * (P + P');
    Z_hist(:, k+1) = compute_meas_and_jacobian(X_hist(:, k+1), m1, m2);
end
end

function dX = calc_dynamics(X, ug, m1, m2)
    dX = zeros(8, 1);
    dX(1) = X(3);
    dX(2) = X(4);
    dX(3) = (-(X(5)*X(1)) - X(6)*(X(1)-X(2)) - X(7)*X(3) - X(8)*(X(3)-X(4))) / m1 - ug;
    dX(4) = (-X(6)*(X(2)-X(1)) - X(8)*(X(4)-X(3))) / m2 - ug;
end

function A = calc_analytical_jacobian(X, m1, m2)
    A = zeros(8, 8);
    A(1, 3) = 1; A(2, 4) = 1;
    A(3, :) = [-(X(5)+X(6))/m1, X(6)/m1, -(X(7)+X(8))/m1, X(8)/m1, -X(1)/m1, -(X(1)-X(2))/m1, -X(3)/m1, -(X(3)-X(4))/m1];
    A(4, :) = [X(6)/m2, -X(6)/m2, X(8)/m2, -X(8)/m2, 0, -(X(2)-X(1))/m2, 0, -(X(4)-X(3))/m2];
end

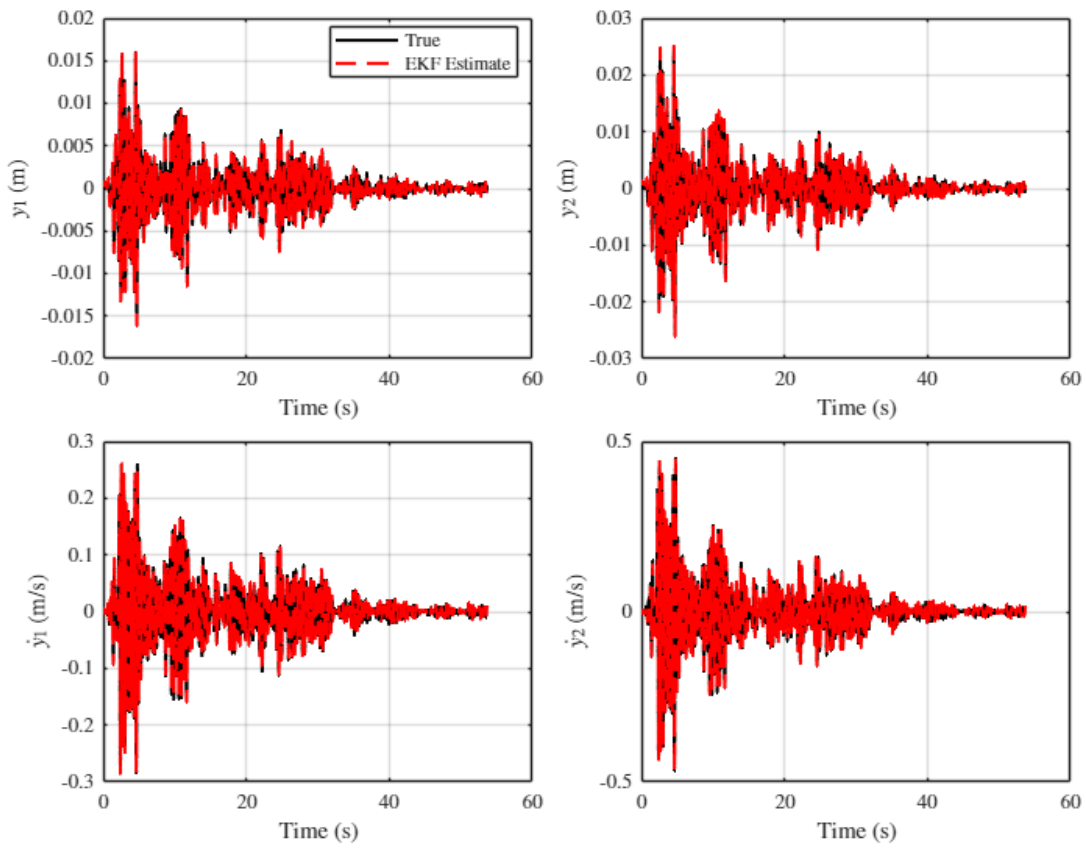
function [Z, H] = compute_meas_and_jacobian(X, m1, m2)
    Z = [ (-(X(5)*X(1)) - X(6)*(X(1)-X(2)) - X(7)*X(3) - X(8)*(X(3)-X(4))) / m1 ;
          (-X(6)*(X(2)-X(1)) - X(8)*(X(4)-X(3))) / m2 ];
    H = zeros(2, 8);
    H(1, :) = [-(X(5)+X(6))/m1, X(6)/m1, -(X(7)+X(8))/m1, X(8)/m1, -X(1)/m1, -(X(1)-X(2))/m1, -X(3)/m1, -(X(3)-X(4))/m1];
    H(2, :) = [X(6)/m2, -X(6)/m2, X(8)/m2, -X(8)/m2, 0, -(X(2)-X(1))/m2, 0, -(X(4)-X(3))/m2];
end

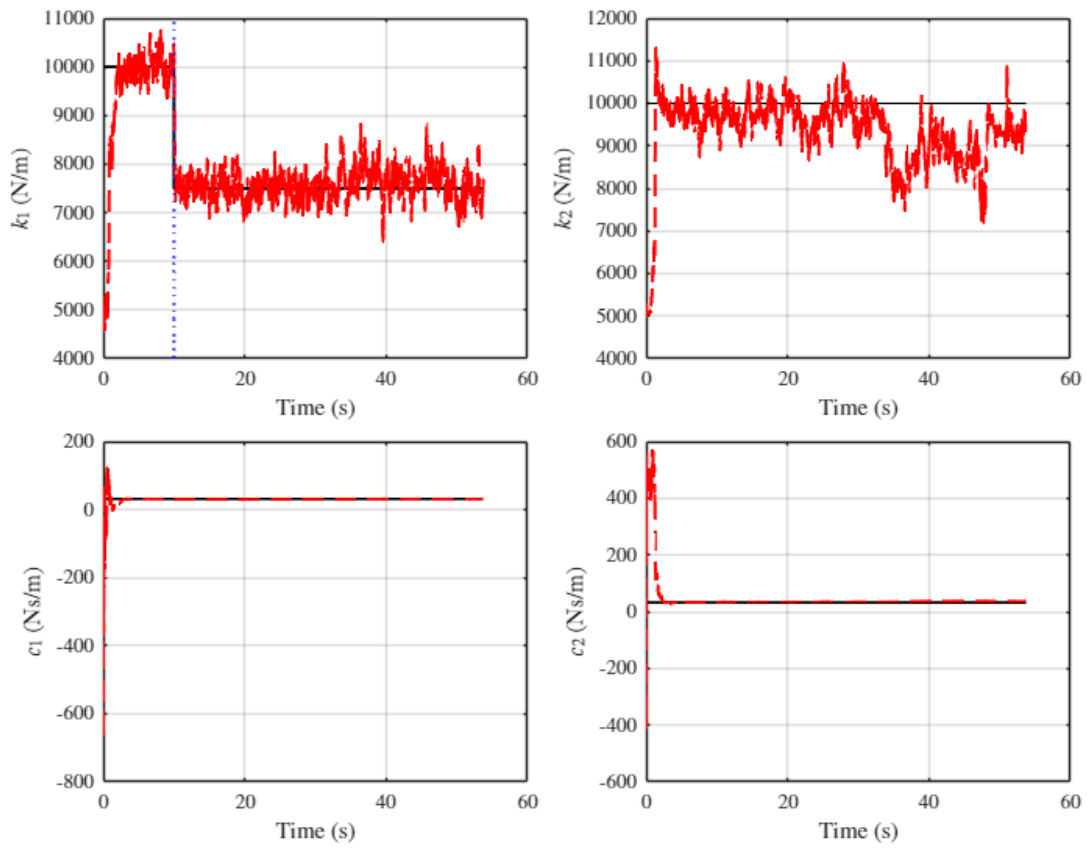
function P_mat = get_P_matrix(choice)
    options = {
        [1e-2 1e-2 1e-2 1e-2 1e10 1e10 1e10 1e10],
        [1e-6 1e-6 1e-6 1e-6 1e6 1e6 1e6 1e6],
        [1e-6 1e-6 1e-6 1e-6 1e4 1e4 1e4 1e4],
        [1e-10 1e-10 1e-8 1e-8 1e8 1e8 1e4 1e4]
    };
    P_mat = diag(options{choice});

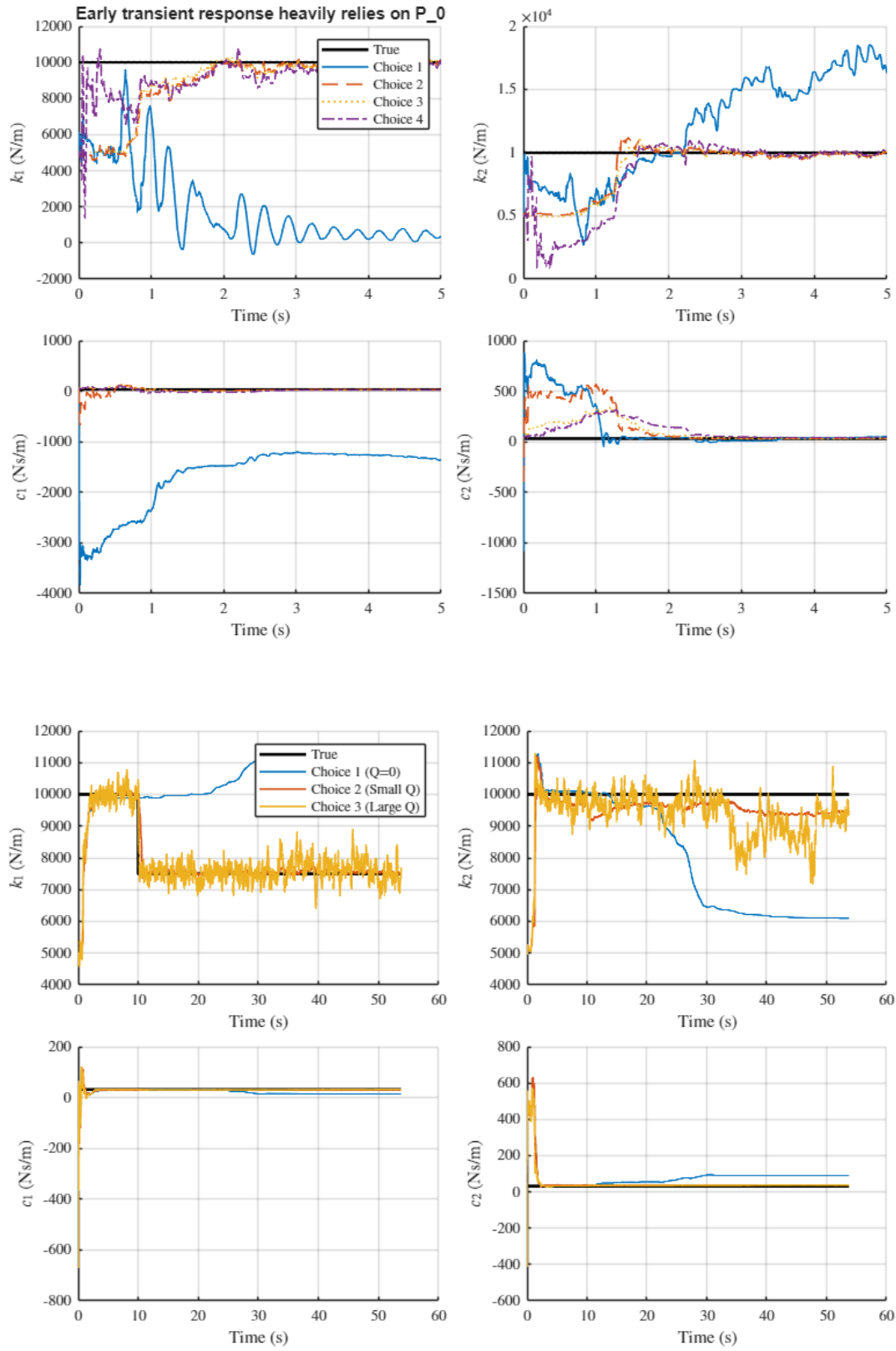
```

end

```
function Q_mat = get_Q_matrix(choice)
    options = {
        zeros(1,8),
        [0 0 0 0 50 50 5e-6 5e-6],
        [0 0 0 0 5000 5000 5e-4 5e-4]
    };
    Q_mat = diag(options{choice});
end
```







Published with MATLAB® R2025b